

Learn C#.Net Core From Scratch With Real World Examples

...

Mohd Manzoor Ahmed [MCT]



manzoor_trainer



manzoorthetrainer.com



ManzoorTM
.....★The Trainer

Prerequisite

- You should know how to use computer.
- Install Visual Studio 2019 Community Edition
<https://www.visualstudio.com/thank-you-downloading-visual-studio/?sku=Community&rel=15>
- Operating System : Windows 10 version 1507 or higher, Windows 8.1 (with Update 2919355), Windows Server 2012 R2, Windows 7 SP1
- Processor : Intel - 1.8 GHz or faster processor. Dual-core or better recommended
- RAM : 2 GB of RAM; 4 GB of RAM recommended
- Hard Disk Space :up to 130 GB of available space, depending on features installed; typical installations require 20-50 GB of free space.

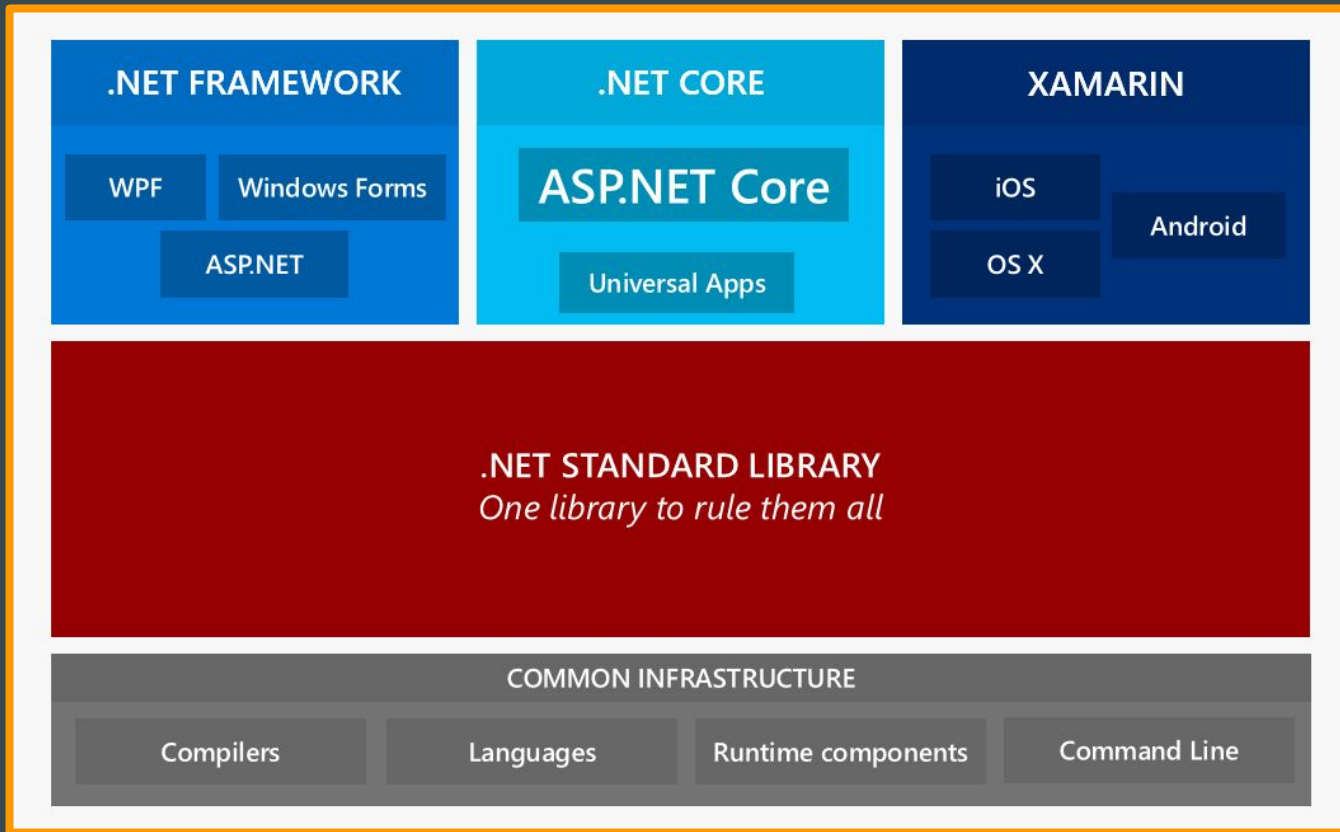
.Net Flavours

- .Net Framework
- .Net Core (Our Course Focus)

What is .Net ?

- What can we do with .Net?
 - We can develop applications like Console App, Windows App, Web App, Mobile App, Web services, Windows services, etc.,
- Why .Net?
 - Because it is a language and OS neutral platform.
- What is .Net?
 - It is a language and OS neutral platform on which we can develop any kind of applications.

.NET Framework Vs .NET Core



**.NET Core
3.0 with
Desktop
Packs**

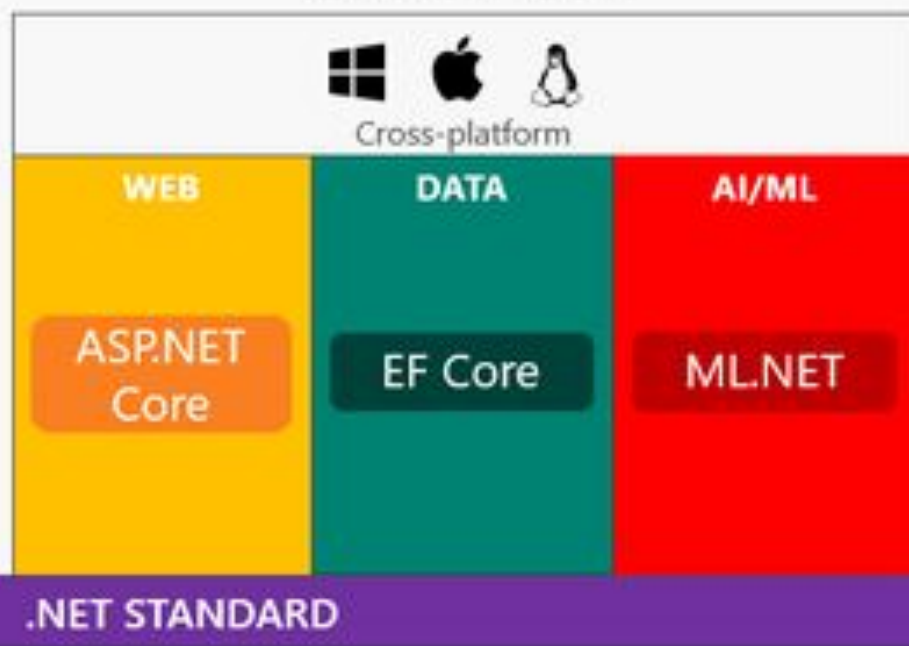
**Install .NET
Core updates
per your
needs**

Desktop Packs

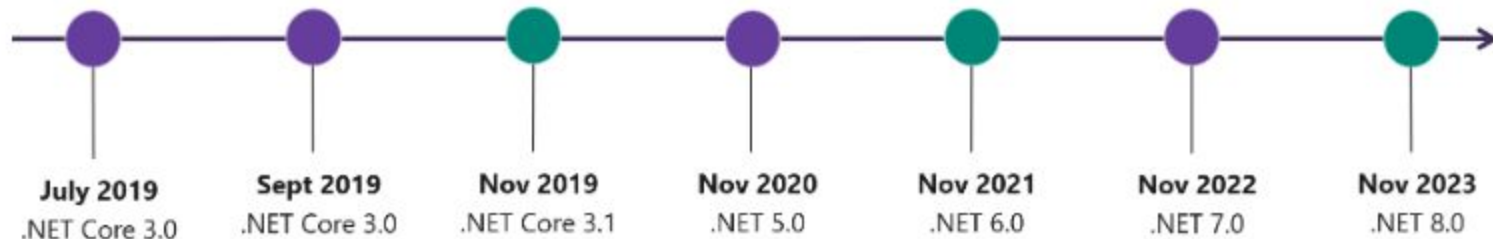


+

.NET Core 3



.NET Schedule



- .NET Core 3.0 release in September
- .NET Core 3.1 = Long Term Support (LTS)
- .NET 5.0 release in November 2020
- Major releases every year, LTS for even numbered releases
- Predictable schedule, minor releases if needed

.NET Framework 4.6

- It is our old framework enhanced that we have been using for more than a decade.
- It is not open source
- It can target only windows OS
- It has all the components like Windows Form Application, ASP.Net Web Forms, ASP.Net MVC, ASP.Net Web API, ASP.Net SignalR, ASP.Net Web Pages, WPF, WCF, etc.,

.NET Core X.X (1, 2, 3, ...)

- It is a new framework similar to .net framework maintained by Microsoft and the .NET community on GitHub.
- It is open source.
- It can target Windows, Linux & Mac OS.
- It does not have all the components, It just has **ASP.Net Core** and **Universal Windows App.(.Net Core 2.X)**

*“**.Net** was language neutral,
.Net Core is OS neutral.”*

- www.ManzoorTheTrainer.com



ManzoorTM
.....★The Trainer

Language Neutral

- Same class library with different syntax
 - `Console.WriteLine(L"C++ Hello, World!");`
 - `Console.WriteLine("C# Hello, World!");`
 - `Console.WriteLine ("VB Hello, World!");`

Get Started With Our First C# Program

- Write a program in any text editor say notepad.
- Compile the program from developer command prompt.
- Run the program. ((csc /target:library Test.s for dll))

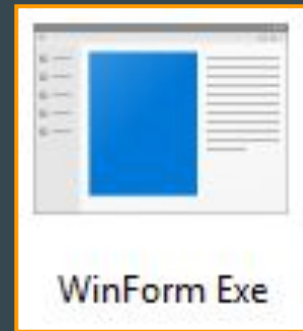
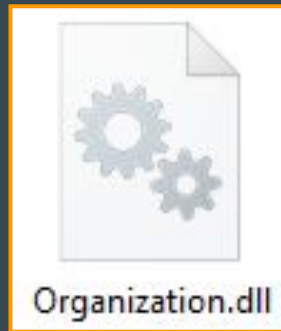


Demo

Let's see a demo on C# First Program

Assemblies

- Assemblies are the building blocks of .NET Framework applications; they form the fundamental unit of deployment.
- An Assembly is a logical unit of code.
- Assembly physically exist as DLLs or EXEs.
- Every assembly file contains:
 - Information about itself called as Assembly Manifest.
 - Information about classes called as Metadata.



Demo

Let's see a demo on **Assemblies**

Getting Started With Visual Studio 2019

- Getting familiar with projects targeting and visual studio 2019.
- Write a hello world program.
- Understand Console.WriteLine() string formatting.
- Understand Console.ReadLine() string parsing.
- Writing a simple program for arithmetic operations.
- Understand Type Casting.
- Understanding how to debug a program.

Demo

Let's see a demo on **Getting Started**

Data Types - Value Type

- Signed integral: sbyte, short, int, long
- Unsigned integral: byte, ushort, uint, ulong
- characters: char
- floating point: float, double
- Boolean: bool
- Enum types
- Struct types
- Nullable value types

Note : Ranges : <https://msdn.microsoft.com/en-us/library/s3f49ktz.aspx>

Data Types - Ref Type

- string
- Array
- Class : object
- Interface

Control Structures

- Conditional Statements
 - If
 - If else
 - Else if ladder
 - Switch case default
- UnConditional Statements
 - Goto
 - Continue
 - break
 - return

- Iterative Statements
 - For
 - While
 - Do while
 - foreach

Demo

Let's see a demo on **Control Structures**

Assignment

Write a C# program which takes current meter reading (CMR), previous meter reading (PMR) and type of connection as input and displays the bill amount.

- Domestic consumers:
 - 0-200 units: Rs 5 per unit
 - 201-300: Rs 7.20 per unit
 - 301-400: Rs 8.50 per unit
 - 401-800: Rs 9 per unit
 - Above 800 units: Rs 9.50 per unit
- Non-domestic/Commercial:
 - 0-100 units: Rs 7.50 per unit
 - 101-300: Rs 8.90 per unit
 - 301-500: Rs 9.40 per unit
 - Above 500: Rs 10 per unit

Arrays

- It is a user defined collection of homogeneous data type elements.
- We use index to access an element of an array.
- When declaring an Array, specify the type, name, dimensions, and size.
- `int[ ] myArray = { 5, 10, 15 };`
- `int[ ] myArray = new int[3];`
 - `myArray[0]=5;`
 - `myArray[1]=10;`
 - `myArray[2]=15;`
- `Int[,] my2dArray = new int[2,3];`

Name	Value
myArray	{int[3]}
[0]	5
[1]	10
[2]	15
my2dArray	{int[3, 2]}
[0, 0]	0
[0, 1]	0
[1, 0]	0
[1, 1]	0
[2, 0]	0
[2, 1]	0

Demo

Let's see a demo on **Arrays & foreach Loop**

Structures

- It is a user defined collection of heterogeneous data type elements.
- We use period operator to access an element of the structure.

```
struct Student
{
    public int StudentId;
    public string Name;
    public double Avg;
}
class Program
{
    static void Main(string[] args)
    {
        Student S;
        S.StudentId = 1234;
        S.Name = "Manzoor";
        S.Avg = 78.9;

        Console.WriteLine("SID : " + S.StudentId);
        Console.WriteLine("Name : " + S.Name);
        Console.WriteLine("Avg : " + S.Avg);
    }
}
```

 S	{MyFirstCSharpProgram.Student}
 S.Avg	78.9
 S.Name	☞ "Manzoor"
 S.StudentId	1234

Demo

Let's see a demo on Structures

var Vs dynamic

- **var** is static typed. It identifies the variable type at compile time itself.
 - var type is fixed with initialization
 - var x="ManzoorTheTrainer";
- **dynamic** is a runtime dispatch type. It identifies the variable type at runtime.
 - dynamic x="ManzoorTheTrainer";
 - dynamic type can changes where it is required

Demo

Let's see a demo on `var` Vs `dynamic`

Object Oriented Programming Through C#

- Objects

- **Jack**
- **Peter**
- **John**

Class

Properties

Methods

Student

- rollNo
- name
- marks

- getDetails()
- getResult()

Object & Class

- **Object**

- Any real time entity is called as an object.
- Every object consist of state(look and feel) and behaviour(what is does).
- States are called as **properties** and behaviors are called as **methods**.

- **Classes**

- Class is a blueprint of an object.
- It consist of fields which are not allowed to access from outside the class.
- It also consist of methods which are allowed to access from outside the class.

Defining A Class

- Defining A Class
- Defining Fields
- Defining Methods
- Public and Private specifiers

Methods

```
class Student
{
    int StudentId;
    string Name;
    double M1, M2;

    public void CreateStudent(int sid, string sname)
    {
        StudentId = sid;
        Name = sname;
    }

    public void SetMarks(double m1, double m2)
    {
        M1 = m1;
        M2 = m2;
    }

    public void Result()
    {
        double avg = (M1 + M2) / 2.0;

        if (avg < 35)
        {
            Console.WriteLine("Fail");
        }
        else
        {
            Console.WriteLine("Pass");
        }
    }
}
```

Fields

.com

Demo

For Defining A Class

Object Creation

- Creating Object
- Accessing Method

```
Student S; //Creating Reference  
S = new Student(); // Creating Object  
  
Student S = new Student(); //Creating Reference & Object  
  
var S = new Student();// Same as above creation
```

```
S.CreateStudent(1, "Manzoor");  
S.SetMarks(78, 89);  
S.Result();
```

Name	Value	Type
🔧 S1	{Student}	Student
🔧 M1	78	double
🔧 M2	89	double
🔧 Name	"Manzoor" 🔍	string
🔧 StudentId	1	int

Demo

For Object Creation

Constructor

- Why do i need constructor?
- Creating a constructor

```
class Student
{
    int StudentId;
    string Name;
    double M1, M2;

    //Constructor
    public Student(int sid, string sname)
    {
        StudentId = sid;
        Name = sname;
    }

    public void SetMarks(double m1, double m2) ...

    public void Result() ...
}
```

```
// Initializing the object
var S = new Student(1, "Manzoor");
S.SetMarks(78, 89);
S.Result();
```

```
// It is not allowed
S.Student(2, "Peter");
```

Constructor

- It's a special method with same name as class name
- Usually constructor is public.
- It won't have any return type.
- It may or may not have parameters.
- Constructor with no parameters is called as default constructor.
- Constructor is invoked automatically when we create an object.
- You cannot invoke constructor explicitly.
- It is used to initialize an object.

Demo

For Constructor

Constructor Overloading

- If we have more than one constructor with different parameters then we call it as constructor overloading

```
class Student
{
    int StudentId;
    string Name;
    double M1, M2;
    string Contact;
    //Default Constructor
    public Student()
    {
        Console.WriteLine("This is default constructor");
    }
    //Constructor
    public Student(int sid, string sname)
    {
        StudentId = sid;
        Name = sname;
    }
    //Constructor
    public Student(int sid, string sname, string cont)
    {
        StudentId = sid;
        Name = sname;
        Contact = cont;
    }

    public void SetMarks(double m1, double m2)
    {
    }

    public void Result()
    {
    }
}
```

```
// Initializing the object
var S1 = new Student(1, "Manzoor");
var S2 = new Student(2, "Peter", "9878765453");
```

Demo

For Constructor Overloading

this Keyword

- this keyword is a default object of current class.
- In what ever class we use “this” keyword it acts like its object
- ‘this’ keyword has two uses.
- ‘this’ keyword is used to refer the members of same class.
- It is also used to invoke a constructor from another constructor of the same class.

```
public Student(int StudentId, string Name)
{
    this.StudentId = StudentId;
    this.Name = Name;
}

public Student(int StudentId, string Name, string Contact)
{
    this.StudentId = StudentId;
    this.Name = Name;
    this.Contact = Contact;
}
```

```
public Student(int StudentId, string Name)
{
    this.StudentId = StudentId;
    this.Name = Name;
}

public Student(int StudentId, string Name, string Contact)
: this(StudentId, Name)
{
    //this.StudentId = StudentId;
    //this.Name = Name;
    this.Contact = Contact;
}
```


Demo

For this Keyword

Method Overloading Compile Time or Static Polymorphism

- If a class contains more than one method with same name and different parameters then we call this as method overloading

```
public void Result()
{
    double avg = (M1 + M2) / 2.0;

    if (avg < 35)
    {
        Console.WriteLine("Fail");
    }
    else
    {
        Console.WriteLine("Pass");
    }
}

public void Result(int minAvgToPass)
{
    double avg = (M1 + M2) / 2.0;

    if (avg < minAvgToPass)
    {
        Console.WriteLine("Fail");
    }
    else
    {
        Console.WriteLine("Pass");
    }
}
```

```
S1.Result();
S1.Result(60);
```

Demo

For Method Overloading

Properties

- Properties are special kind of methods to access any field in a secured way.
- It contains two kind of methods i.e., getter and setter.
- Getter method is to read the value of the private field.
- Setter method is to write the value of the private field.
- It looks like we are accessing the fields directly.

But, internally it works as methods where we can perform some kind of validations.

```
class Student
{
    int StudentId;
    string Name;
    double _M1, _M2;
    string Contact;

    public double M1 {
        get { return _M1; }
        set { _M1 = value; }
    }

    public double M2
    {
        get { return _M2; }
        set {
            if (value >= 0)
            {
                _M2 = value;
            }
        }
    }
}
```

```
S.M1 = 78;
S.M2 = 89;
```

Demo

For Properties

Latest Style Of Class

- Properties are special kind of methods to access any field in a secured way.
- It contains two kind of methods i.e., getter and setter.
- Getter method is to read the value of the private field.
- Setter method is to write the value of the private field.
- It looks like we are accessing the fields directly. But, internally it works as methods where we can perform some kind of validations.
- With auto properties
- Latest style of object creation

```
//New Style Of Class
class Student
{
    public int StudentId { get; set; }
    public string Name { get; set; }
    public string Contact { get; set; }
    public double M1 { get; set; }
    public double M2 { get; set; }
}
```

```
//New Style Of Object Creation
var S = new Student() { StudentId = 1, Name = "Manzoor", Contact = "9878765654" };
```

Demo

For Latest Class Style

Static Variables

- Any member of a class declared with the keyword `static` is called as static member.
- Memory for static variable is allocated only once and is shared by all the objects of that class.
- We can access static variables with directly with the class name if they are public.

```
class HousingLoanCustomer
{
    int CustomerId;
    string Name;
    double LoanAmount;
    public static double _ROI;
}
class Program
{
    static void Main(string[] args)
    {
        HousingLoanCustomer._ROI = 11.5;
    }
}
```


Demo

For Static Variables

Static Constructor

- Static constructor is a constructor which is declared with the keyword as static.
- Static constructor is use to initialize static variables (private).
- Static constructor does not have any access modifiers (i.e,Public,Private).
- Static constructor is parameter less.
- There can be one and only one static constructor in a class.
- Static Constructor is invoked before creation of any object.

```
class HousingLoanCustomer
{
    int CustomerId;
    string Name;
    double LoanAmount;
    static double _ROI;

    static HousingLoanCustomer()
    {
        _ROI = 11.2;
    }
}
```

Demo

For Static Constructor

Static Methods

- Static method is a method which is declared with the keyword static.
- There can be n number of static methods in a class.
- Static methods can access only static variables.
- Static method can be directly accessed by Class name.

```
public static void Enquiry(double EnLoanAmount, int months)
{
    Console.WriteLine("ROI :" + ROI);
    Console.WriteLine("Loan Amount:" + EnLoanAmount);
    Console.WriteLine("IntAmount:" + EnLoanAmount * ROI / 100);
    Console.WriteLine("RepayAmount:" + (EnLoanAmount + (EnLoanAmount * ROI / 100)));
    Console.WriteLine("EMI Months:" + months);
    Console.WriteLine("EMIs:" + (EnLoanAmount + (EnLoanAmount * ROI / 100)) / months);
}
```

Demo

For Static Methods

Static Class

- Static class is a class which is declared with static keyword.
- Static class should contain only static methods.
- Static class does not allow creation of class reference or object.
- Like `Console` is a predefined static class which contains all static methods like `WriteLine()`

```
static class Calculate
{
    public static int Add(int a, int b)
    {
        return a + b;
    }
    public static int Sub(int a, int b)
    {
        return a - b;
    }
    public static int Mul(int a, int b)
    {
        return a * b;
    }
}
```

```
//Cannot declare a variable of static type
Calculate C;

//Cannot create an instance of static class
C = new Calculate();

int result = Calculate.Add(24, 78);
Console.WriteLine("The sum is " + result);
```

Demo

For Static Classes

Namespaces

- Namespaces are logical group of classes and namespaces.
- It is used for avoiding name clashes between two sets of code.

```
namespace Admin
{
    class Employee
    {
        public void Add() { }
        public void Update() { }
    }
}

namespace HR
{
    class Employee
    {
        public void CalculateSalary() { }
        public void CalculateLeaves() { }
    }
}
```

```
var E1 = new Admin.Employee();
E1.Add();
E1.Update();
```

```
var E2 = new HR.Employee();
E2.CalculateSalary();
E2.CalculateLeaves();
```

```
namespace MyOrg
{
    namespace Admin
    {
        class Employee
        {
            public void Add() { }
            public void Update() { }
        }
    }
}
```

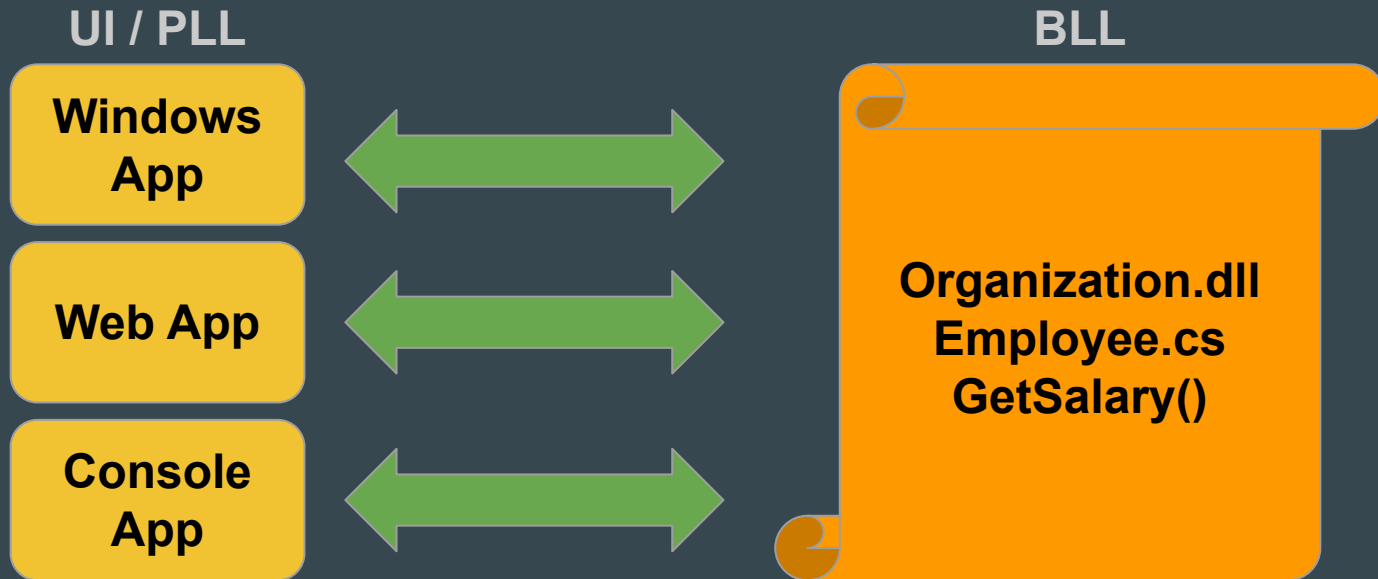
```
var E = new MyOrg.Admin.Employee();
```

```
using MyOrg.Admin;
class Program
{
    static void Main(string[] args)
    {
        var E = new Employee();
    }
}
```


Demo

For Namespaces

Reusable Component Or .dll



Note : Every UI/PLL will have a local copy of dll, any changes made dll will not be reflected automatically

Organization.dll

- Create a new project **Organization**.
- Create a class **Employee**
- Create a method **GetSalary()**
- Create a Console UI
- Add dll reference
- Add namespace
- Create Object of **Employee**
- Call **GetSalary**
- **Public and non public class**

```
namespace Organization
{
    public class Employee
    {
        public int EmpId { get; set; }
        public string Name { get; set; }
        public double SalaryPerDay { get; set; }
        public double HRA { get; set; }

        public double GetSalary(int NoOfLeavesTaken)
        {
            int CurrentYear = DateTime.Now.Year;
            int CurrentMonth = DateTime.Now.Month;
            int NoOfWorkingDays = DateTime.DaysInMonth(CurrentYear, CurrentMonth);
            int NoOfDaysPresent = NoOfWorkingDays - NoOfLeavesTaken;
            double BasicSalary = NoOfDaysPresent * SalaryPerDay;
            double GrossSalary = BasicSalary + (BasicSalary * (HRA / 100.0));
            return GrossSalary;
        }
    }
}
```

```
using Organization;
class Program
{
    static void Main(string[] args)
    {
        var E = new Employee()
        {
            EmpId = 1,
            Name = "Manzoor",
            SalaryPerDay = 400,
            HRA = 12
        };

        double GS = E.GetSalary(3);
        Console.WriteLine("Gross salary is " + GS);
    }
}
```

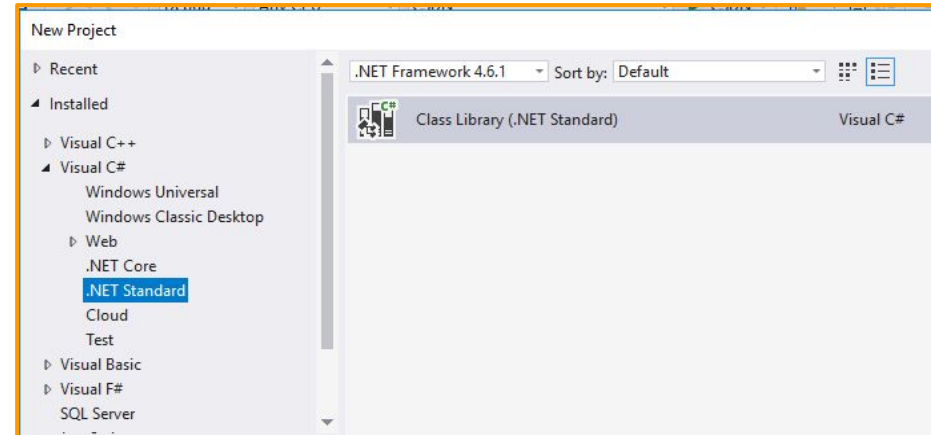
Demo

For dll Consumption in ConsoleUI

.Net Standard Library

- Let's use Organization.dll in Windows Classic App.
- Let's Create .Net standard library.
- .Net standard library can consumed by both Classic .Net and .Net Core Apps.

	Code	Description	Project
✖		Project '..\MySalaryDLL\Organization.csproj' targets 'netcoreapp2.0'. It cannot be referenced by a project WindowsFormsApp1 that targets '.NETFramework,Version=v4.6.1'.	
⚠		The project 'Organization' cannot be referenced. The referenced project is targeted to a different framework family (.NETCoreApp)	WindowsFormsApp1



Demo

For .Net Standard Library

Inheritance

- Deriving a new class from an existing class is called Inheritance.
- Existing class is called as base class.
- New class is called as derived class.
- Everything from the base class gets inherited except private members.

```
//Base Class
class Customer
{
    public int CustomerId { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
}
```

IS - A

```
//Child Class
class BankCustomer : Customer
{
    public int AccountNumber { get; set; }
    public double Balance { get; set; }
}
```

Demo

For Inheritance

Types Of Inheritances

- **Single Inheritance** : If a derived class has a single base class then it is called as single inheritance i.e., A(Base) → B (Derived)
- **Multilevel Inheritance** : If a class is derived from a class which is already a derived class then it is called as multi level inheritance. i.e., A(Base) → B (Derived) → C(Derived)
- **Multiple Inheritance** : If a class is derived from more than one class then it is called as multiple inheritance. i.e., A,B(Base) → C(Derived)
- C# does not support multiple inheritance using classes.

```
//Base Class
class Customer
{
    public int CustomerId { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
}
```

//Is A Relationship

```
//Child Class
class BankCustomer : Customer
{
    public int AccountNumber { get; set; }
    public double Balance { get; set; }
}
```

Single

```
class Customer
{
    public int CustomerId { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
}
```

```
class BankCustomer : Customer
{
    public int AccountNumber { get; set; }
    public double Balance { get; set; }
}
```

```
class LoanCustomer : BankCustomer
{
    public double LoanAmount { get; set; }
    public double ROI { get; set; }
}
```

Multilevel

Demo

For Types Of Inheritance

Protected Members

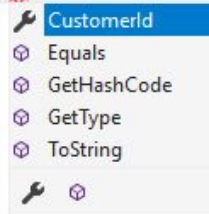
- Protected members of a classes can be inherited in the child class.
- Protected members of a class cannot be accessed from outside of a class.

```
//Base Class
class Customer
{
    public int CustomerId { get; set; }
    private string Name { get; set; }
    protected string Email { get; set; }
}

//Child Class
class BankCustomer : Customer
{
    //Public and Protected Members Get Inherited
    //public int CustomerId { get; set; }
    //protected string Email { get; set; }
    public int AccountNumber { get; set; }
    public double Balance { get; set; }
}
```

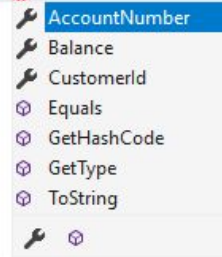
```
var C = new Customer();
```

C.



```
var C = new BankCustomer();
```

C.



Demo

For Protected Members

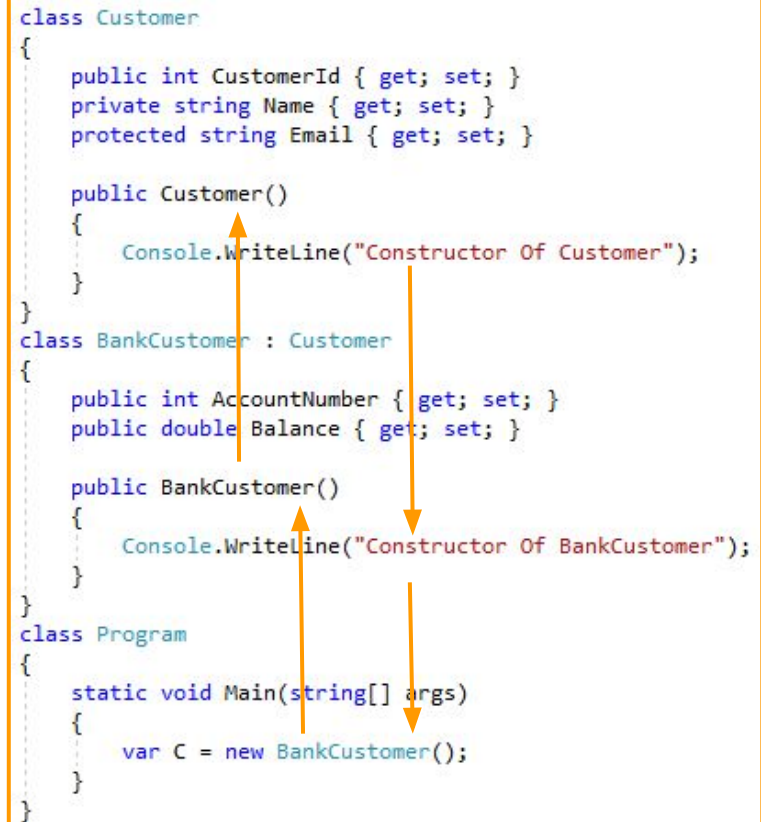
Constructor Chaining (Or) base keyword

- Any constructor of child class always invokes default constructor of base class.
- Calling the base class constructor from the child class' constructor using **base keyword** is called as constructor chaining.
- base keyword is a default object of its base class.
- It is also used to refer the members of base class from derived class.

```
class Customer
{
    public int CustomerId { get; set; }
    private string Name { get; set; }
    protected string Email { get; set; }

    public Customer()
    {
        Console.WriteLine("Constructor Of Customer");
    }
}
class BankCustomer : Customer
{
    public int AccountNumber { get; set; }
    public double Balance { get; set; }

    public BankCustomer()
    {
        Console.WriteLine("Constructor Of BankCustomer");
    }
}
class Program
{
    static void Main(string[] args)
    {
        var C = new BankCustomer();
    }
}
```



```
C:\Windows\system32\cmd.exe
Constructor Of Customer
Constructor Of BankCustomer
Press any key to continue . . .
```

```

class Customer
{
    public int CustomerId { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
    public Customer()
    {
        Console.WriteLine("Constructor Of Customer");
    }
    public Customer(int CustomerId, string Name, string Email)
    {
        Console.WriteLine("P - Constructor Of Customer");
        this.CustomerId = CustomerId;
        this.Name = Name;
        this.Email = Email;
    }
}

class BankCustomer : Customer
{
    public int AccountNumber { get; set; }
    public double Balance { get; set; }
    public BankCustomer()
    {
        Console.WriteLine("Constructor Of BankCustomer");
    }
    public BankCustomer(int CustomerId, string Name, string Email,
        int AccountNumber, double Balance)
    {
        Console.WriteLine("P - Constructor Of BankCustomer");
        this.CustomerId = CustomerId;
        this.Name = Name;
        this.Email = Email;
        this.AccountNumber = AccountNumber;
        this.Balance = Balance;
    }
}

class Program
{
    static void Main(string[] args)
    {
        var C = new BankCustomer(1, "Manzoor", "info@manzoorthetrainer", 3432324, 67000);
    }
}

```

```

class Customer
{
    public int CustomerId { get; set; }
    public string Name { get; set; }
    public string Email { get; set; }
    public Customer()
    {
        Console.WriteLine("Constructor Of Customer");
    }
    public Customer(int CustomerId, string Name, string Email)
    {
        Console.WriteLine("P - Constructor Of Customer");
        this.CustomerId = CustomerId;
        this.Name = Name;
        this.Email = Email;
    }
}

class BankCustomer : Customer
{
    public int AccountNumber { get; set; }
    public double Balance { get; set; }
    public BankCustomer()
    {
        Console.WriteLine("Constructor Of BankCustomer");
    }
    public BankCustomer(int CustomerId, string Name, string Email,
        int AccountNumber, double Balance):base(CustomerId,Name,Email)
    {
        Console.WriteLine("P - Constructor Of BankCustomer");
        //this.CustomerId = CustomerId;
        //this.Name = Name;
        //this.Email = Email;
        this.AccountNumber = AccountNumber;
        this.Balance = Balance;
    }
}

class Program
{
    static void Main(string[] args)
    {
        var C = new BankCustomer(1, "Manzoor", "info@manzoorthetrainer", 3432324, 67000);
    }
}

```

Method Overriding

- If a method in the derived class has the same signature as that of in the base class with different implementation then we say that method in the derived class overrides the method of base class.
- This process is called as Method Overriding.
- The method of the base class should be declared as **virtual** and method in derived class should be declared as **override**.

```

class Customer
{
    Fields and Initialization

    public virtual void Details()
    {
        Console.WriteLine("Customer Id :" + CustomerId);
        Console.WriteLine("Name :" + Name);
        Console.WriteLine("Email :" + Email);
    }
}

class BankCustomer : Customer
{
    Fields and Initialization

    public override void Details()
    {
        Console.WriteLine("Customer Id :" + CustomerId);
        Console.WriteLine("Name :" + Name);
        Console.WriteLine("Email :" + Email);
        Console.WriteLine("AccountNumber :" + AccountNumber);
        Console.WriteLine("Balance :" + Balance);
    }

    public void Deposit(double amount)
    {
        Balance = Balance + amount;
    }
}

static void Main(string[] args)
{
    var C = new BankCustomer(1, "Manzoor", "info@manzoor", 1001, 7500);
    C.Details();
    C.Deposit(5000);
    C.Details();
}

```


Other Use Of base

- **base** keyword is also used to refer the members of base class from derived class.

```
class Customer
{
    Fields and Initialization

    public virtual void Details()
    {
        Console.WriteLine("Customer Id :" + CustomerId);
        Console.WriteLine("Name :" + Name);
        Console.WriteLine("Email :" + Email);
    }
}

class BankCustomer : Customer
{
    Fields and Initialization

    public override void Details()
    {
        //Console.WriteLine("Customer Id :" + CustomerId);
        //Console.WriteLine("Name :" + Name);
        //Console.WriteLine("Email :" + Email);
        base.Details();
        Console.WriteLine("AccountNumber :" + AccountNumber);
        Console.WriteLine("Balance :" + Balance);
    }

    public void Deposit(double amount)
    {
        Balance = Balance + amount;
    }
}
```

```
static void Main(string[] args)
{
    var C = new BankCustomer(1, "Manzoor", "info@manzoor", 1001, 7500);
    C.Details();
    C.Deposit(5000);
    C.Details();
}
```


Demo

For Method Overriding

Sealed Method & Class

- A sealed method is a method which is declared with the keyword sealed and it cannot be overridden by the derived class.
- But we can invoke this method.
- Sealed keyword is always used in combination with override keyword.
- A **sealed class** is the class which is declared with the keyword sealed and it cannot be derived by other classes.
- But we can create the objects of sealed class

```
class Customer
{
    Fields and Initialization

    public virtual void Details()...
}
class BankCustomer : Customer
{
    Fields and Initialization

    public override sealed void Details()...

    public void Deposit(double amount)...
}
class LoanCustomer:BankCustomer
{
    public override sealed void Details()...
```

Code	Description
CS0239	'LoanCustomer.Details()': cannot override inherited member 'BankCustomer.Details()' because it is sealed

```
sealed class A...

class B : A |
```

class CSDN.B

'B': cannot derive from sealed type 'A'

Show potential fixes (Alt+Enter or Ctrl+.)

Demo

For Sealed Methods And Classes

Abstract Method

- An abstract method is a method which is declared with the keyword abstract.
- It does not have body.
- It should be implemented by the derived class.
- If a method is abstract then the class should be abstract.

```
abstract class A
{
    public void Show()...
    public abstract void Display();
}
class B : A
{
    public override void Display()...
```

Demo

For Abstract Method

Abstract Class

- It is the blueprint of the problem.
- It says what to do but not how to do.
- An abstract class is the class which is declared with the keyword `abstract`.
- Basically it contains at least one abstract method.
- It's object cannot be created.
- But it's reference can be created.
- It is used to implement runtime polymorphism.

```
abstract class A
{
    public void Show()...
    public abstract void Display();
}

class B : A
{
    public override void Display()...
}

class Program
{
    static void Main(string[] args)
    {
        A a;
        a = new B();
        a.Show();
        a.Display();
    }
}
```

Demo

For Abstract Class

Runtime or Dynamic Polymorphism

- Deriving the abstract class by one or more classes and overriding all the abstract methods.
- Creating the reference of abstract class and making it an object of derived class based on user's input.
- And calling the method of abstract class.
- The method gets referred from derived class at runtime based on user input.
- This complete process is called as runtime polymorphisms.

Runtime or Dynamic Polymorphism

```
abstract class Employee
{
    public int EmpId { get; set; }
    public string Name { get; set; }
    public double SalaryPerHour { get; set; }
    public virtual void Details()
    {
        Console.WriteLine("EmpId : " + EmpId);
        Console.WriteLine("Name : " + Name);
        Console.WriteLine("SalaryPerHour : " + SalaryPerHour);
    }
    public abstract void CalculateSalary(int NoOfHours);
}
```

```
class SoftwareEngineer : Employee
{
    public string Technology { get; set; }
    public override void Details()
    {
        base.Details();
        Console.WriteLine("Technology : " + Technology);
    }
    public override void CalculateSalary(int NoOfHours)
    {
        //15% HRA
        double BasicSalary = SalaryPerHour * NoOfHours;
        double HRA = BasicSalary * 0.15;
        double GrossSalary = BasicSalary + HRA;
        Console.WriteLine("Software Engineer's Gross salary is " + GrossSalary);
    }
}
```

```
class TeamLead : Employee
{
    public int TeamSize { get; set; }
    public override void Details()
    {
        base.Details();
        Console.WriteLine("Team Size : " + TeamSize);
    }
    public override void CalculateSalary(int NoOfHours)
    {
        //28% HRA
        double BasicSalary = SalaryPerHour * NoOfHours;
        double HRA = BasicSalary * 0.28;
        double GrossSalary = BasicSalary + HRA;
        Console.WriteLine("Team Lead's Gross salary is " + GrossSalary);
    }
}
```

Runtime or Dynamic Polymorphism

```
static void Main(string[] args)
{
    Employee E;
    int Type = int.Parse(Console.ReadLine());
    switch (Type)
    {
        case 1:
            E = new SoftwareEngineer() { EmpId = 1, Name = "Manzoor", SalaryPerHour = 25, Technology = "MS.Net" };
            break;

        case 2:
            E = new TeamLead() { EmpId = 100, Name = "Peter", SalaryPerHour = 45, TeamSize = 23 };
            break;
        default:
            E = new SoftwareEngineer() { EmpId = 1, Name = "Manzoor", SalaryPerHour = 25, Technology = "MS.Net" };
            break;
    }
    E.Details();
    E.CalculateSalary(36);
}
```

Demo

For Runtime Polymorphism

Interface

- An interface is a pure abstract class.
- It is a reference type and it contains only abstract members.
- Any implementation must be placed in class that implements them.
- All the member declarations inside interface are implicitly public and abstract.
- We can implement multiple inheritance using interfaces.

```
interface IOperations
{
    void GetAll();
    void GetById();
    void Insert();
    void Update();
    void Delete();
}
```

```
class Employees : IOperations
{
    public void Delete()...
    public void GetAll()...
    public void GetById()...
    public void Insert()...
    public void Update()...
}
```

Demo

For Interfaces

Partial Class

- Partial class is a class which is declared with partial keyword.
- Its implementation can be spread over multiple files.
- Multiple classes prefixed with partial keyword with same class name and different methods in different files, acts as a single class with all the methods.

```
partial class A
{
    public void Show() { }
}

partial class A
{
    public void Display() { }
}

class Program
{
    static void Main(string[] args)
    {
        A a = new A();
        a.Show();
        a.Display();
    }
}
```

Demo

For Partial Classes

Struct vs Classes

- Structs Only:
 - Cannot support inheritance
 - Are value types
 - Are passed by value (like integers)
 - Cannot have a null reference (unless Nullable is used)
 - Do not have a memory overhead per new instance - unless 'boxed'
- Classes Only:
 - Can support inheritance
 - Are reference (pointer) types
 - The reference can be null
 - Have memory overhead per new instance
- Both Classes and Structs:
 - Are compound data types typically used to contain a few variables that have some logical relationship
 - Can contain methods and events
 - Can support interfaces

Collection Classes

- Collection is an alternate way of storing multiple variable number of elements or objects.
- Like Stack, Queue, ArrayList , etc., are collection classes present in **System.Collection** namespace
- There is no type safety in collection classes i.e., in a collection object of a Queue we can store few integers and few strings.
- We need to type cast the elements while accessing it.

```
Stack stack = new Stack(2);  
stack.Push(12);  
stack.Push("Manzoor");
```

```
stack.Pop();//Manzoor  
stack.Pop();//12
```

```
Queue queue = new Queue();  
queue.Enqueue(13);  
queue.Enqueue("Peter");
```

```
queue.Dequeue();//13  
queue.Dequeue();//Peter
```

```
ArrayList arrayList = new ArrayList();  
arrayList.Add(1);  
arrayList.Add("John");  
arrayList.Add("123");
```

```
arrayList[0].ToString();//1  
arrayList[1].ToString();//John  
arrayList[2].ToString();//123
```

```
arrayList.Remove("John");
```

```
arrayList[0].ToString();//1  
arrayList[1].ToString();//123
```

Demo

For Collection Classes

Generic Collection Classes

- Generic collection is an alternate way of storing multiple variable number of elements or objects.
- Like Stack, Queue, **List**, etc., are generic collection classes present in **System.Collection.Generic** namespace
- There is type safety in generic collection classes i.e., in a generic collection object of a Queue we cannot store few integers and few strings.
- Hence it is strongly typed.
- We need not to type cast the elements while accessing it.

```
Stack<int> stack = new Stack<int>(2);
stack.Push(12);
stack.Push("Manzoor");

stack.Pop(); //Manzoor
stack.Pop(); //12

Queue<string> queue = new Queue<string>();
queue.Enqueue(13);
queue.Enqueue("Peter");

queue.Dequeue(); //13
queue.Dequeue(); //Peter

List<string> List = new List<string>();
List.Add(134);
List.Add("John");
List.Add("123");

List[0].ToString(); //134
List[1].ToString(); //John
List[2].ToString(); //123

List.Remove("John");

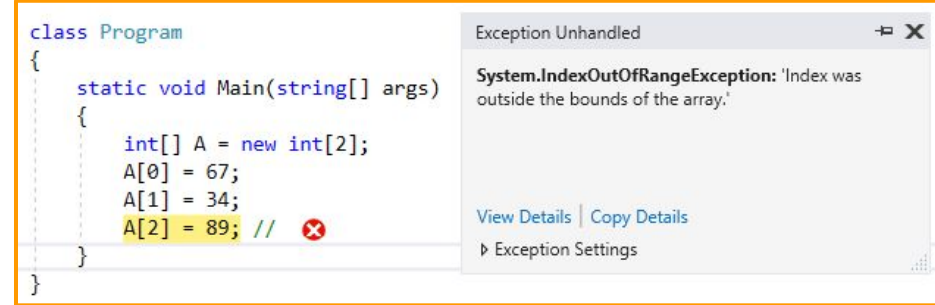
List[0].ToString(); //134
List[1].ToString(); //123
```

Demo

For Generic Collection Classes

Exception Handling

- Runtime error is called as an exception.
- Exception handling is an in-built mechanism in .NET core to detect and handle runtime errors.
- C# provides three keywords **try**, **catch** and **finally** to do exception handling.
- The try encloses the statements that might throw an exception.
- The catch handles an exception if one exists.
- The finally can be used for doing any clean up process.



```
try
{
    int[] A = new int[2];
    A[0] = 67;
    A[1] = 34;
    //It will throughh an exception here
    A[2] = 89;
}
catch (Exception E)
{
    Console.WriteLine(E.Message);
}
```

Demo

For Exception Handling

Thanks